More practice - Dynamic programming and memoization

5 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	3
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Dynamic programming and memoization | 5 questions | tap a title to jump to it

M	1. Maximize sum without choosing adjacent elements	PracHub	2026-01-06
M	2. Fence Painting Minimum Operations (Gemini Team)	1Point3Acres	2026-04-27
H	3. Determine winner in stack-merging game	PracHub	2026-01-06
H	4. Tree Distance Sum (Re-root DP)	1Point3Acres	2026-01-19
H	5. Count Perfect Wake Numbers up to N	1Point3Acres	2026-05-07

STUDY PATH: question 105 of 167 (more practice)

Priority score 3.29 | Dynamic programming and memoization | asked 287 days ago

MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 1 of 5

★ TOP 75: rank 61 in the Medium and Hard priority list

★ Maximize sum without choosing adjacent elements

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | **ALL TIME 57.2** | **30 DAYS 0** | **PRIORITY 50/228**

ABOUT THIS QUESTION

Topic: More practice - Dynamic programming and memoization

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #198 House Robber

LeetCode match: LC 198 House Robber (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

QUESTION

Overview: This question evaluates a candidate's understanding of array-based optimization and dynamic programming concepts for computing a maximum sum under non-adjacent selection constraints.

APPROACH

dp: take = skip_prev + x, skip = max(take_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximize sum without choosing adjacent
# elements
#
# This question evaluates a candidate's understanding of
# array-based optimization and dynamic programming
# concepts for computing a maximum sum under non-adjacent
# selection constraints.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
#
# BRUTE FORCE (say it first): Recursion over take or skip
# At index i either skip it (go to i + 1) or take it and
# jump to i + 2, and keep the larger total. Exact and the
# oracle; the same suffixes are solved over and over. Time
# O(2^n). Space O(n).
```

```python
# BRUTE FORCE (say it first): Recursion over take or skip
# | time O(2^n)
def _house_brute(nums):
    def go(i):
        if i >= len(nums):
            return 0
        return max(go(i + 1), nums[i] + go(i + 2))

    return go(0)


# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take: best when the current element is taken,
```

```python
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last are neighbours: ignore one of
    # them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i onward, filled from the
    # right
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] == dp[i] and nums[i] > 0:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
```

```python
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_gap(nums, k):
    # taking a house blocks the next k houses (k = 1 is
    # the
    # classic problem): dp[i] = max(dp[i + 1], nums[i] +
    # dp[i + k + 1])
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    values = [2, 7, 9, 3, 1]
    assert sum(values[i] for i in houses) == total
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12
    # blocking two houses after each pick
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

FUNCTION AND TEST CASES

Signature

```python
def solution(nums: list[int]) -> int:
```

Sample call

```python
solution([2, 7, 9, 3, 1])
```

Test cases

Input	Expected
`([2, 7, 9, 3, 1],)`	`12`
`([1, 2, 3, 1],)`	`4`

STUDY PATH: question 106 of 167 (more practice)

Priority score 2.5 | Dynamic programming and memoization | asked 176 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 2 of 5

Fence Painting Minimum Operations (Gemini Team)

ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 200/228**

ABOUT THIS QUESTION

Topic: More practice - Dynamic programming and memoization

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

EXAMPLES

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]
      (covers row 1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still
      missing row 2)
Optimal answer: 3.
```

APPROACH

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether vertical strokes paint one plank fully, whether horizontal strokes must cover consecutive planks at one height, and the sizes involved.

BRUTE FORCE TO SAY OUT LOUD

Try all combinations of strokes. Exponential.

OPTIMAL IDEA

Divide and conquer on the minimum height: for a range with base height h, painting every plank vertically costs the number of planks; otherwise paint one horizontal band up to the minimum height m (cost 1) and recurse on the segments above m, split where planks are exactly m. The answer is the minimum of the two. O(n^2) worst case, O(n log n) with a sparse table or a min-stack for range minimum queries.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 3. All equal heights (1). Strictly increasing heights. A single plank.

LIKELY FOLLOW-UPS

A brush of width W, or return the strokes.

PITFALLS

Forgetting the vertical option, so a tall single plank is overcounted. Off-by-one on the band height.

HOW TO THINK ABOUT IT

The shortest plank in a range decides the shape: a horizontal band along the bottom, up to that plank's height, is free of regret because every plank in the range needs it. That splits the range at the shortest planks, so recurse on the pieces above the band. The other choice for a range is to paint every plank vertically. Take whichever is cheaper.

Basis: Codeforces 448C, Painting Fence. Strokes: a vertical stroke paints one whole plank; a horizontal stroke paints one row across consecutive planks that all reach that height.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

TIME	SPACE
O(n^2)	O(n)
worst case	recursion

WHY IT IS SLOW

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

TIME	SPACE
O(n log n)	O(n log n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

FOLLOW-UP 2 Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Fence Painting Minimum Operations (Gemini
# Team)
#
# Examples: n = 5, a = [2, 2, 1, 2, 1] Optimal: 1)
# Horizontal stroke at h=1 across [1..5] (covers row 1
# across whole fence) 2) Horizontal stroke at h=2 across
# [1..2] 3) Vertical stroke at plank 4 (height 2 still
# missing row 2) Optimal answer: 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The horizontal brush is only W planks wide.
# 2. Return the strokes, not just the count.
# 3. Is it the same as largest rectangle in a histogram?
#
# INTERVIEW NOTES
# The shortest plank in a range decides the shape: a
# horizontal band along the bottom, up to that plank's
# height, is free of regret because every plank in the
# range needs it. That splits the range at the shortest
# planks, so recurse on the pieces above the band. The
# other choice for a range is to paint every plank
# vertically. Take whichever is cheaper.
#
# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time
# For a range with base height b: either paint every plank
# vertically (cost = number of planks) or paint a
# horizontal band up to the range minimum (cost = min - b)
# and recurse on the pieces split at the minimum planks.
# Finding the minimum by scanning makes a sorted or
# constant fence quadratic. Time O(n^2) worst case. Space
# O(n) recursion.


# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time | time O(n^2) worst case
def paint_brute(a):
    def solve(l, r, base):
        if l > r:
            return 0
        m = min(a[l : r + 1])
        cost = m - base
        i = l
        while i <= r:
            if a[i] == m:
                i += 1
                continue
            j = i
            while j <= r and a[j] != m:
                j += 1
            cost += solve(i, j - 1, m)
            i = j
        return min(r - l + 1, cost)

    return solve(0, len(a) - 1, 0)


# HELPERS for the base question
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table


# FOLLOW-UP 2: Return the strokes, not just the count.
# Answer: Record which option won in each range and
# rebuild the list by walking the recursion again:
# horizontal bands as (row, l, r), vertical strokes as
# (plank).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
```

```
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)


# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]


if __name__ == "__main__":
    # the example: 3 strokes
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
```

```
    assert paint([3]) == 1
    # a flat fence is one stroke per row
    assert paint([4, 4, 4, 4]) == 4
    # tall thin planks are cheaper to paint upright
    assert paint([5, 0, 5]) == 2
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3
    # a narrow brush needs several strokes per row
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2
```

NOTES FROM CANDIDATES

Classic recursive divide-and-conquer: for range [l..r] with base height h:
Option A: paint every plank vertically -> cost r - l + 1.
Option B: paint a horizontal band of height min - h, then recurse on the sub-ranges above the band split by planks at height min.
Take the minimum.
Equivalent to Cartesian-tree DP; runs in O(N log N) with min-stack or sparse table, or O(N²) with the recursive form.
Verbalize the recursion's correctness: the band optimally covers all planks >= min in the range, then each "taller column block" is solved independently.

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

STUDY PATH: question 107 of 167 (more practice)

Priority score 2.25 | Dynamic programming and memoization | asked 287 days ago

HARD | PracHub | Technical Screen | 3 of 5

Determine winner in stack-merging game

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 31.2 | 30 DAYS 0 | PRIORITY 121/228

ABOUT THIS QUESTION

Topic: More practice - Dynamic programming and memoization

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #877 Stone Game

LeetCode match: Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-14)

Original: https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

QUESTION

Overview: This question evaluates understanding of combinatorial game theory, impartial game analysis, state-space encoding, and efficient state-space search techniques.

APPROACH

Impartial game: encode the state (tuple of stack sizes), memoize win/lose with DFS over all legal merges, or use Sprague-Grundy values. Prune with symmetry (sorted state).

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy of this question has only the overview, so before anything ask for the exact rules: what a move is (for example merge two stacks, possibly adjacent ones, with a size limit), who moves first, whether the player who cannot move loses (normal play) or wins, and the number and size of stacks.

BRUTE FORCE TO SAY OUT LOUD

Recursive search over every legal move with no memory. Exponential.

OPTIMAL IDEA

Treat it as an impartial game. Encode the state as the multiset of stack sizes (a sorted tuple, which removes symmetric duplicates). A state is winning if some move leads to a losing state, and a state with no moves is losing for the player to move under normal play. Memoize on the sorted tuple. If the game splits into independent parts, use Sprague-Grundy numbers (mex of reachable values) and XOR them instead of searching the product.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No legal moves, one stack, two equal stacks, a state reachable by two move orders (checks memoization).

LIKELY FOLLOW-UPS

Larger states (find a pattern from small cases with a brute-force table, then prove it), misere play, or counting winning first moves.

PITFALLS

Starting to code before the rules are confirmed. Memoizing on an unsorted state and blowing up the state space.

HOW TO THINK ABOUT IT

Two-player game with perfect play and no draws: define win(state) = some move leads to a state where the opponent loses. A state with no moves is a loss for whoever must move. If a move does not care about stack order, sort the state so equal positions share one cache entry. If the game splits into independent parts, use Grundy numbers and XOR them.

Basis: Written. The statement is not available in my copy, so this uses an example rule (confirm the real one): any two stacks may be merged if their combined size is at most a cap; a player who cannot move loses. The method (memoized win/lose search, Sprague-Grundy for independent games) is the part that carries over.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(state):
    for nxt in all_merges(state):
        if not wins(nxt):
            return True  # opponent loses
    return False  # no move wins
```

TIME	SPACE
exponential in the number... exponential in the number of stacks	**O(depth)**

WHY IT IS SLOW

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

OPTIMAL SOLUTION · Memoize on the sorted state

TIME	SPACE
O(number of distinct sorted states * k^2)	**O(number of states)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

FOLLOW-UP 2 Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

FOLLOW-UP 3 Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Determine winner in stack-merging game
#
# This question evaluates understanding of combinatorial
# game theory, impartial game analysis, state-space
# encoding, and efficient state-space search techniques.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The game is a sum of independent games (several
# separate boards or piles).
# 2. Misere play (the player who cannot move wins).
# 3. Count the number of winning first moves.
#
# INTERVIEW NOTES
# Two-player game with perfect play and no draws: define
# win(state) = some move leads to a state where the
# opponent loses. A state with no moves is a loss for
# whoever must move. If a move does not care about stack
# order, sort the state so equal positions share one cache
# entry. If the game splits into independent parts, use
# Grundy numbers and XOR them.
#
# BRUTE FORCE (say it first): Plain recursion over every
# move
# Try every merge, and the current player wins if some
# merge leaves the opponent in a losing position. Correct,
# but the same positions are explored again and again
# through different move orders. Time exponential in the
# number of stacks. Space O(depth).


from functools import lru_cache


# BRUTE FORCE (say it first): Plain recursion over every
# move | time exponential in the number of stacks
def merge_game_brute(stacks, cap):
    def wins(state):
        for i in range(len(state)):
            for j in range(i + 1, len(state)):
                if state[i] + state[j] <= cap:
                    nxt = (
                        state[:i]
                        + state[i + 1 : j]
                        + state[j + 1 :]
                        + [state[i] + state[j]]
                    )
                    if not wins(nxt):
                        return True
        return False

    return wins(list(stacks))


# HELPERS for the base question
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new


# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
        # state
        return any(not wins(nxt) for nxt in options)

    return wins(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def grundy(stacks, cap):
    # Sprague-Grundy number of one game (mex of its moves)
    @lru_cache(None)
    def g(state):
        reach = {g(nxt) for nxt in moves(state, cap)}
        number = 0
        while number in reach:
            number += 1
        return number
```

```
    return g(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def sum_of_games_wins(boards, cap):
    # independent boards: XOR their Grundy numbers
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0


# FOLLOW-UP 3: Count the number of winning first moves.
# Answer: Keep the same memoized win/lose function and
# count the moves whose resulting state is a loss for the
```

```
# opponent.
def winning_first_moves(stacks, cap):
    # how many different first moves win?
    state = tuple(sorted(stacks))
    return sum(1 for nxt in moves(state, cap)
               if not first_player_wins(nxt, cap))


if __name__ == "__main__":
    # one stack: no move, so the first player loses
    assert not first_player_wins([3], 5)
    # two stacks that can merge: one move ends it, so we
    # win
    assert first_player_wins([1, 1], 5)
    # three stacks: after one merge the other two may
    # merge too
    assert first_player_wins([1, 1, 1], 2) is True
    # in misere play the result flips for a single stack
    assert first_player_wins([3], 5, misere=True)
    assert winning_first_moves([1, 1, 1], 2) >= 1
    # normal play: a position wins exactly when Grundy !=
    # 0
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)
    # two separate boards: XOR of their numbers
    both = sum_of_games_wins([[1, 1], [1, 1]], 5)
    assert both is False
```

FUNCTION AND TEST CASES

Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

Test cases

Input	Expected
`['Y']`	`False`
`['Y', 'W']`	`True`

ALSO REPORTED · 1 more

Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API. Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".

Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).

Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.

Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.

Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

STUDY PATH: question 108 of 167 (more practice)

Priority score 2.22 | Dynamic programming and memoization | asked 274 days ago | frequency Single report

HARD | IN APP LC 834 | 1Point3Acres | onsite-coding | 4 of 5

Tree Distance Sum (Re-root DP)

ASKED 2026-01-19 | 274 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 124/228

ABOUT THIS QUESTION

Topic: More practice - Dynamic programming and memoization

Optimal answer from: your app's interview approach (LeetCode #834)

LeetCode match: LC 834 Sum of Distances in Tree

Frequency: Single report | Last asked: 2026-01-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

APPROACH

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes. Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

HOW TO THINK ABOUT IT

Running a BFS from every node costs O(n^2). Notice that moving the root from a parent p to its child v changes every distance by exactly one: the nodes inside v's subtree get closer by 1, all the others get farther by 1. So compute the answer for one root, then derive each child's answer from its parent's in O(1) using the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = [0] * n
    for s in range(n):
        dist = bfs(adj, s)
        ans[s] = sum(dist)
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n BFS runs of O(n) each, O(n^2). Moving the root across one edge changes every distance by exactly 1, so two passes over the tree give all answers in O(n).

OPTIMAL SOLUTION · Two passes: subtree sizes, then re-root

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, v) + 1. If w is outside it, dist(w, v) = dist(w, p) + 1. There are size[v] nodes of the first kind and n - size[v] of the second, so ans[v] = ans[p] - size[v] + (n - size[v]).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Tree Distance Sum (Re-root DP)
#
# Onsite coding round 1: given an undirected tree on N
# nodes, return for every node the sum of distances to all
# other nodes. Equivalent to LC 834. Interviewer expects
# baseline O(n²) DFS followed by the O(n) re-root DP.
# Input: undirected tree with N nodes and N-1 edges.
# Output: array ans where ans[u] is the sum of shortest-
# path distances from u to every other node. Walk through
# baseline (BFS/DFS from every node, O(n²)) before
# optimizing. Final solution should be O(n) via two-pass
# re-root DP. Examples: Edges:
# [[0,1],[0,2],[2,3],[2,4],[2,5]] ans: [8, 12, 6, 10, 10,
# 10]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the re-root formula correct?
#
# INTERVIEW NOTES
# Running a BFS from every node costs O(n^2). Notice that
# moving the root from a parent p to its child v changes
```

```
# every distance by exactly one: the nodes inside v's
# subtree get closer by 1, all the others get farther by
# 1. So compute the answer for one root, then derive each
# child's answer from its parent's in O(1) using the
# subtree sizes.
#
# BRUTE FORCE (say it first): BFS from every node
# Run a BFS from each node and add up the distances.
# Correct and the right baseline to state in the first
# minutes; quadratic is too slow for 10^5 nodes. Time
# O(n^2). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def tree_sums_brute(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    out = []
```

```
    for s in range(n):
        dist, q = {s: 0}, deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    q.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# BASELINE (say it first): BFS from every node
def distance_sums_bfs(n, edges):
    # the baseline to say first: a BFS from every node,
    # O(n^2)
    adj, out = build(n, edges), []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
def tree_distance_sums(n, edges):
```

```
    if n == 0:
        return []
    adj = build(n, edges)
    # order: parents before children (no recursion)
    parent, order = [-1] * n, [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    # size[u]: nodes in u's subtree
    # inside[u]: distances from u to its own subtree
    size, inside = [1] * n, [0] * n
    for u in reversed(order):
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]
    # the root's answer is already complete
    ans = [0] * n
    ans[0] = inside[0]
    # moving the root from p to child u: the size[u] nodes
    # below get 1 closer, the other n - size[u] get 1
    # farther
    for u in order[1:]:
        ans[u] = ans[parent[u]] + n - 2 * size[u]
    return ans


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(6, edges) == want
    assert distance_sums_bfs(6, edges) == want
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    # a chain of 100000 nodes: no recursion trouble
    n = 100000
    chain = [(i, i - 1) for i in range(1, n)]
    first = tree_distance_sums(n, chain)[0]
    assert first == n * (n - 1) // 2
```

NOTES FROM CANDIDATES

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.
Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].
Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

PREPARATION

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.
Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

STUDY PATH: question 109 of 167 (more practice)

Priority score 2.0 | Dynamic programming and memoization | asked 166 days ago | frequency Single report

HARD | 1Point3Acres | phone-screen, onsite-coding | 5 of 5

Count Perfect Wake Numbers up to N

ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 187/228**

ABOUT THIS QUESTION

Topic: More practice - Dynamic programming and memoization

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

QUESTION

EXAMPLES

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9
    is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is
    not a valley because its right neighbor 3 is
    smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both
    strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

APPROACH

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact definition: no zero digit, all digits distinct, no interior digit smaller than both neighbors. Ask for the range of N and whether to count 1..N inclusive.

BRUTE FORCE TO SAY OUT LOUD

Test every number from 1 to N with a digit check. O(N * d), fine for small N.

OPTIMAL IDEA

Digit DP over positions with state (used-digit mask, last digit, whether the digits have started falling, tight flag). Numbers with fewer digits than N are counted without a tight constraint, and zero is never allowed, so there are no leading-zero cases. Digits must rise then fall, which is exactly the no-valley rule. There is also a closed form of C(9, k) * 2^(k - 1) for k-digit numbers.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

196, 23, 12463 perfect; 1546, 320, 34321 not perfect. N = 9, N = 10, N a power of ten.

LIKELY FOLLOW-UPS

A range [A, B] (inclusion-exclusion), or different digit rules.

PITFALLS

Forgetting the zero rule. Letting a digit repeat across the tight and free branches.

HOW TO THINK ABOUT IT

Distinct digits with no zero and no valley means the digits go up and then down (they can only change direction once). That structure is both a closed form for each length and a state for a digit DP: position, which digits are used, the last digit, and whether the digits have started falling. The tight flag handles the bound N.

Basis: Digit DP, like LeetCode 1012 (Numbers With Repeated Digits) and 233. Definition from the report: no zero digit, all digits distinct, and no interior digit is a valley (strictly smaller than both neighbours).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

TIME	SPACE
O(N * d)	O(1)

WHY IT IS SLOW

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

OPTIMAL SOLUTION · Part 1: test a single number

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

TIME	SPACE
O(d * 2^9 * 9 * 9)	O(2^9 * 9)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Perfect Wake Numbers up to N
#
# Examples: 196 -> perfect (digits 1,9,6 all distinct, no
# 0, 9 is not a valley) 23 -> perfect 12463 -> perfect (1<
2,
# 2<4, 4>6? No, 4<6, 6>3 — 6 is not a valley because its
# right neighbor 3 is smaller) 1546 -> not perfect (4 is
# adjacent to 5 and 6, both strictly larger) 320 -> not
# perfect (contains 0) 34321 -> not perfect (digit 3
# repeats)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the DP only need a 'falling' flag instead of
# the full history?
# 2. How do you count the numbers in a range [A, B]?
# 3. What is the closed form without a DP?
# 4. Is there a closed form for k-digit numbers?
# 5. Count perfect numbers in a range [A, B].
#
# INTERVIEW NOTES
# Distinct digits with no zero and no valley means the
# digits go up and then down (they can only change
# direction once). That structure is both a closed form
# for each length and a state for a digit DP: position,
# which digits are used, the last digit, and whether the
# digits have started falling. The tight flag handles the
# bound N.
#
# BRUTE FORCE (say it first): Test every number from 1 to
# N
# Apply the three rules to each number. Correct and the
# oracle; it is hopeless for N around 10^9. Time O(N * d).
# Space O(1).


from functools import lru_cache
from math import comb


# BRUTE FORCE (say it first): Test every number from 1 to
# N | time O(N * d)
def count_perfect_brute(N):
    return sum(
        1 for x in range(1, N + 1) if is_perfect(x)
    )
```

```
# HELPERS for the base question
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# Answer: Pick the k distinct digits from 1..9 (C(9, k)
# ways); the largest must be the peak, and each of the
# other k - 1 digits goes either left or right of it in
# sorted order, so 2^(k-1) arrangements: C(9, k) *
# 2^(k-1).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0


# BASE QUESTION (optimal): test a single number
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))
```

```python
# PART 2: count the perfect numbers up to N
# FOLLOW-UP 2: How do you count the numbers in a range [A,
# B]?
# Answer: Use inclusion-exclusion: count(B) - count(A -
# 1). The DP is a function of the upper bound only, and it
# is cheap enough (a few thousand states) to call twice.
def count_up_to(n):
    if n < 1:
        return 0
    digits = list(map(int, str(n)))
    size = len(digits)
    # all shorter numbers: closed form
    total = sum(with_k_digits(k) for k in range(1, size))
```

```python
    @lru_cache(None)
    def go(pos, mask, last, falling, tight):
        # count completions of a number of full length
        if pos == size:
            return 1
        out = 0
        limit = digits[pos] if tight else 9
        for d in range(1, limit + 1):
            if mask >> d & 1:
                continue
            fall = falling
            if pos:
                if d > last and falling:
                    # going up after going down: a valley
                    continue
                if d < last:
                    fall = True
            out += go(pos + 1, mask | 1 << d, d, fall,
                      tight and d == digits[pos])
        return out

    # numbers with the same length as n, not above n
    return total + go(0, 0, 0, False, True)


# FOLLOW-UP 5: Count perfect numbers in a range [A, B].
# Answer: Compute count(B) - count(A - 1); the DP depends
# on the upper bound only.
def count_between(a, b):
    # inclusion-exclusion over the upper bounds
    return count_up_to(b) - count_up_to(a - 1)


if __name__ == "__main__":
    for ok in (196, 23, 12463):
        assert is_perfect(ok)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    # the DP agrees with checking every number
    for n in (0, 9, 10, 99, 100, 999, 5000, 20000):
        every = range(1, n + 1)
        brute = sum(is_perfect(i) for i in every)
        assert count_up_to(n) == brute
    assert count_between(10, 99) == (
```

```python
        count_up_to(99) - count_up_to(9))
    # 1-digit and 2-digit counts from the closed form
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72
```

NOTES FROM CANDIDATES

Part 1 is a linear scan; Part 2 is digit DP with state (position, mask of used digits, last digit, is_tight against n). Track whether the previous step went up or down to detect a future valley. Multiple candidates report not finishing Part 2 in the 45-minute window. The interviewer is generally patient and dispenses hints ("think DP" / "think digit DP") when you stall.
Watch out: the "no valley" rule is about interior digits, so the first and last digit are never themselves the valley; they can still create one for their neighbor.
The full prompt is short and recent — expect the follow-up to vary (e.g. count exactly-K-digit Perfect Wake Numbers).

PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.